

AI휴먼 과정 - Day 10

Pandas DataFrame 기초 - CSV 읽기, 데이터 구조 확인, 기초통계

오늘의 핵심 질문: "CSV 파일을 받았을 때 분석 전에 무엇부터 확인해야 하는가?"



오늘의 학습목표

- 1 Pandas가 데이터 분석에서 어떤 역할을 하는지 설명할 수 있다.
- 2 Series와 DataFrame의 차이를 설명할 수 있다.
- 3 CSV 파일을 `pd.read_csv()`로 읽어 DataFrame으로 만들 수 있다.
- 4 `head()`, `tail()`, `info()`, `shape`, `dtypes`로 데이터 구조를 점검할 수 있다.
- 5 `describe()`와 주요 Series 메소드로 기초통계를 확인할 수 있다.
- 6 분석 전 데이터 점검 결과를 간단한 데이터 사전(Data Dictionary) 형태로 정리할 수 있다.

오늘의 전체 흐름

01

Pandas와 표 형태 데이터 이해

02

Series / DataFrame / Index / Column

03

CSV 파일 읽기

04

데이터의 앞·뒤 행 확인

05

행·열 구조와 자료형 확인

06

숫자형 데이터의 기초통계 확인

07

범주형 데이터의 빈도 확인

08

데이터 점검표 만들기

의사코드

1

START

CSV 파일 불러오기

2

데이터 크기와 컬럼 확인

앞부분/뒷부분 확인

3

자료형 확인

기초통계 확인

4

END

데이터 사전 작성

Pandas란?

- Pandas는 Python에서 표 형태 데이터를 다루기 위한 대표적인 데이터 분석 라이브러리이다.
- Excel의 표, CSV, 데이터베이스 조회 결과처럼 "행과 열"로 구성된 데이터를 처리하기 좋다.
- NumPy가 수치 배열 계산에 강하다면 Pandas는 컬럼 이름과 인덱스를 가진 데이터 처리에 강하다.
- 주요 객체: Series, DataFrame

설치

```
pip install pandas
```

불러오기 / 버전 확인

```
import pandas as pd  
print(pd.__version__)
```

용어 정리 ① Pandas 핵심 용어

용어	의미	쉬운 해석
Pandas	표 형태 데이터 분석 라이브러리	CSV/Excel 데이터를 다루는 도구
Series	1차원 데이터 객체	한 개 컬럼과 유사
DataFrame	2차원 표 데이터 객체	여러 컬럼으로 구성된 표
Index	각 행을 식별하는 레이블	행 번호 또는 행 이름
Column	열을 식별하는 이름	변수명, 항목명
dtype	컬럼 데이터 타입	정수, 실수, 문자열 등의 형식

Series 이해하기

- Series는 한 줄짜리 "컬럼"처럼 생각할 수 있다.
- 값(values)과 인덱스(index)를 가진다.

핵심

- scores[0]처럼 인덱스를 통해 값에 접근할 수 있다.
- DataFrame에서 한 개 컬럼을 꺼내면 일반적으로 Series가 된다.

샘플 코드

```
import pandas as pd
scores = pd.Series([78, 85, 92], name="Python점수")
print(scores)
print(scores.index)
print(scores.values)
```

DataFrame 이해하기

- DataFrame은 행(row)과 열(column)로 구성된 2차원 표이다.
- 각 컬럼은 서로 다른 dtype을 가질 수 있다.

샘플 코드

```
import pandas as pd

data = {
    "이름": ["민지", "준호", "서연"],
    "Python점수": [88, 76, 95],
    "출석률": [98.0, 92.5, 100.0]
}

df = pd.DataFrame(data)
print(df)
```

Series와 DataFrame 비교

구분	Series	DataFrame
차원	1차원	2차원
형태	하나의 컬럼	여러 컬럼의 표
대표 생성	pd.Series()	pd.DataFrame()
컬럼 선택 결과	df["컬럼"]은 Series	df[["컬럼1", "컬럼2"]]은 DataFrame

샘플

```
one = df["Python점수"]
two = df[["이름", "Python점수"]]
print(type(one))
print(type(two))
```


Index와 Column

- Index: DataFrame의 행을 구분하는 레이블
- Columns: DataFrame의 열 이름 목록
- 기본 Index는 0부터 시작하는 정수 형태이다.

실무 포인트

- 컬럼명은 분석 코드에서 반복해서 사용되므로 오타자를 먼저 확인한다.
- 행 수와 열 수를 함께 확인하면 파일이 예상한 구조인지 빠르게 판단할 수 있다.

샘플 코드

```
print(df.index)
print(df.columns)
```

CSV 란?

- CSV = Comma-Separated Values
- 값을 쉼표(,)로 구분해 저장하는 텍스트 기반 데이터 형식이다.
- Excel 없이도 만들고 읽을 수 있으며 데이터 교환에 널리 사용된다.

주의

- 한글 CSV는 인코딩 문제를 확인해야 한다.

예시

```
학생ID,이름,전공,Python점수  
S001,김민지,컴퓨터공학,88  
S002,박준호,경영학,76
```

CSV 읽기 - pd.read_csv()

기본 형식

```
import pandas as pd
df = pd.read_csv("students.csv")
```

주요 인수

인수	역할	예시
filepath_or_buffer	읽을 파일 경로	"students.csv"
encoding	문자 인코딩	"utf-8", "cp949"
sep	구분자	"," , "\t"
usecols	읽을 컬럼 제한	["이름", "점수"]
nrows	처음 N개 행만 읽기	100
header	헤더 행 지정	0
index_col	인덱스로 사용할 컬럼	"학생ID"

read_csv() 샘플 코드

```
import pandas as pd

# UTF-8 CSV 전체 읽기
df = pd.read_csv("students.csv", encoding="utf-8")

# 필요한 컬럼만 읽기
small_df = pd.read_csv(
    "students.csv",
    usecols=["이름", "Python점수", "AI점수"]
)

# 처음 5개 행만 읽기
sample_df = pd.read_csv("students.csv", nrows=5)

print(small_df)
```

데이터 미리보기 - head(), tail()

- head(n): 위에서 n개 행 확인, 기본값은 5
- tail(n): 아래에서 n개 행 확인, 기본값은 5

왜 확인하는가?

- 컬럼 값이 예상 형태인지 확인
- 헤더가 정상적으로 읽혔는지 확인
- 파일의 처음과 끝에 이상한 행이 없는지 확인

샘플 코드

```
print(df.head())  
print(df.head(3))  
print(df.tail())  
print(df.tail(2))
```

구조 확인 핵심 속성

속성	의미	예시 결과
df.shape	(행 수, 열 수)	(100, 6)
df.ndim	차원 수	2
df.size	전체 원소 수	600
df.columns	컬럼 이름	Index([...])
df.index	행 인덱스	RangeIndex(...)
df.dtypes	컬럼별 자료형	int64, float64, object

샘플

```
rows, cols = df.shape
print("행 수:", rows)
print("열 수:", cols)
```

DataFrame의 dtype 이해

Pandas에서 자주 보는 dtype

dtype	의미	예시
int64	정수형	85
float64	실수형	92.5
object	일반 문자열/혼합 데이터	"서울"
bool	논리형	True / False
datetime64[ns]	날짜·시간형	2026-08-30

info() - 데이터 구조를 한 번에 점검

info()에서 확인하는 것

- 전체 행 범위
- 전체 컬럼 수
- 컬럼명
- 각 컬럼의 Non-Null Count
- 각 컬럼 dtype
- 메모리 사용량

주의

- `df.info()`는 정보를 화면에 직접 출력한다.
- `print(df.info())`를 사용하면 마지막에 `None`이 추가로 출력된다.

샘플 코드

```
print(df.info())
```


describe() - 숫자형 기초통계

샘플 코드

```
print(df.describe())
```

활용

점수, 매출, 수량, 가격 등 숫자형 컬럼을 빠르게 점검한다.

주요 결과

항목	의미
count	값의 개수
mean	평균
std	표준편차
min	최솟값
25%	1사분위수
50%	중앙값
75%	3사분위수
max	최댓값

범주형 데이터 요약

문자열 컬럼은 빈도 확인이 유용하다.

샘플 코드

```
print(df["전공"].value_counts())  
print(df["전공"].nunique())  
print(df["전공"].unique())
```

의미

- `value_counts()`: 값별 출현 횟수
- `nunique()`: 서로 다른 값의 개수
- `unique()`: 고유한 값 목록

예시 질문

"어떤 전공의 학생이 가장 많은가?"

숫자 Series의 주요 집계 메소드

샘플 코드

```
score = df["Python점수"]

print(score.sum())
print(score.mean())
print(score.min())
print(score.max())
print(score.median())
print(score.count())
```

메소드 정리

메소드	의미
sum()	합계
mean()	평균
min()	최솟값
max()	최댓값
median()	중앙값
count()	결측값을 제외한 값의 개수

Pandas 주요 객체·상수·속성·메소드 정리

분류	이름	역할
객체	pd.Series()	1차원 데이터 생성
객체	pd.DataFrame()	2차원 표 데이터 생성
함수	pd.read_csv()	CSV를 DataFrame으로 읽기
특수값	pd.NA	Pandas의 결측값 표현
특수값	pd.NaT	날짜·시간 계열의 결측값 표현
속성	df.shape	행·열 수 확인
속성	df.columns	컬럼 이름 확인
속성	df.index	인덱스 확인
속성	df.dtypes	컬럼별 dtype 확인
메소드	df.head()	앞부분 확인
메소드	df.tail()	뒷부분 확인
메소드	df.info()	구조·Non-Null-dtype 확인
메소드	df.describe()	숫자형 기초통계
메소드	Series.value_counts()	값별 빈도
메소드	Series.nunique()	고유값 개수
메소드	Series.mean()	평균

한 컬럼과 여러 컬럼 선택

한 컬럼 → Series

```
python_score = df["Python점수"]  
print(type(python_score))
```

여러 컬럼 → DataFrame

```
score_df = df[["이름", "Python점수", "AI점수"]]  
print(type(score_df))
```

주의

- 한 컬럼: 대괄호 1쌍
- 여러 컬럼: 컬럼명 리스트가 들어가므로 대괄호가 2쌍

데이터 점검 순서 - 실무 체크리스트

CSV 파일을 처음 받았다면 다음 순서를 권장한다.

01

파일이 정상적으로 읽히는가?

03

shape로 행·열 수가 예상과 같은가?

05

dtypes와 info()로 자료형과 값의 개수를 확인했는가?

07

value_counts()/nunique()로 범주 구성을 확인했는가?

02

head()/tail()로 값 모양이 예상과 같은가?

04

columns로 컬럼명이 정확한가?

06

describe()로 숫자 범위를 확인했는가?

08

컬럼 설명을 데이터 사전으로 남겼는가?

데이터 사전(Data Dictionary)이란?

- 데이터의 각 컬럼이 무엇을 의미하는지 정리한 문서이다.
- 분석자와 협업자가 같은 의미로 데이터를 이해하게 해준다.

예시

컬럼명	dtype	의미	예시값
학생ID	object	학생 식별 코드	S001
Python점수	int64	Python 평가 점수	88
출석률	float64	출석 비율(%)	98.0

대표 실습 의사코드

START

pandas 불러오기

실습 CSV가 없으면 샘플 파일 생성

CSV를 DataFrame으로 읽기

DataFrame 타입 출력

head / tail 출력

shape / columns / index / dtypes 출력

info 출력

한 컬럼을 Series로 선택

여러 컬럼을 DataFrame으로 선택

숫자형 기초통계 출력

범주형 빈도 출력

각 컬럼에 대해

dtype 확인

Non-Null 개수 확인

고유값 개수 확인

데이터 점검표 생성

점검표를 CSV로 저장

END



자주 발생하는 실수와 원인

FileNotFoundError

- 원인: CSV 경로가 잘못됨
- 확인: 현재 작업 폴더와 파일명 확인

UnicodeDecodeError

- 원인: 실제 파일 인코딩과 encoding 인수가 다름
- 확인: utf-8, utf-8-sig, cp949 등 확인

KeyError: '컬럼명'

- 원인: 존재하지 않는 컬럼명을 사용
- 확인: `print(df.columns)`

`print(df.info())`의 마지막 None

- 원인: `info()` 자체가 출력하고 반환값은 None
- 해결: `df.info()`만 호출

`df['A', 'B']` 오류

- 원인: 여러 컬럼 선택 방식 오류
- 해결: `df[["A", "B"]]`



오늘의 정리와 다음 학습 연결

오늘 할 수 있어야 하는 것

- Series와 DataFrame 구분
- CSV → DataFrame 변환
- head(), tail(), shape, columns, dtypes, info()로 구조 파악
- describe()와 집계 메소드로 숫자 데이터 파악
- value_counts(), nunique()로 범주 데이터 파악
- 데이터 사전 형태의 점검표 작성

다음 학습 연결

- 오늘: "데이터가 어떤 상태인지 확인"
- 다음: "분석할 수 있도록 데이터를 정제"
- 다음 핵심: loc/iloc, 결측치, 중복, 자료형 변환